TUGAS GESTURE

Workshop Mobile Applications Framework

E41242361 TIARA DWI AGUSTIA PUTRI GOL F

```dart
import 'package:flutter/material.dart';
import 'home_page.dart';

Run | Debug | Profile
void main() {
  runApp(GestureApp());
}

class GestureApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Gesture Lab',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(useMaterial3: true, colorSchemeSeed:
      home: HomePage(),
    ); // MaterialApp
  }
}
```

MAIN DART

Bagian ini adalah awal dari aplikasi.
Di sini aplikasi dijalankan menggunakan fungsi runApp(). Kemudian dibuat sebuah widget utama bernama GestureApp.

Widget ini berfungsi untuk:

- Mengatur judul aplikasi (Gesture Lab)
- Mengatur tema warna
- Menentukan halaman pertama yaitu HomePage

```dart
import 'package:flutter/material.dart';
import 'pages/tap_page.dart';
import 'pages/drag_page.dart';
import 'pages/scale_page.dart';
import 'pages/playground_page.dart';

class HomePage extends StatelessWidget {
  final menus = [
    {"title": "Tap Interaction", "page": TapPage()},
    {"title": "Drag Movement", "page": DragPage()},
    {"title": "Scale & Zoom", "page": ScalePage()},
    {"title": "Gesture Playground", "page": PlaygroundPage()},
  ];
```

```dart
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Gesture Lab")),
      body: Padding(
        padding: EdgeInsets.all(16),
        child: GridView.builder(
          itemCount: menus.length,
          gridDelegate: SliverGridDelegateWithFixedCrossAxisCount(
            crossAxisCount: 2,
            crossAxisSpacing: 16,
            mainAxisSpacing: 16,
          ), // SliverGridDelegateWithFixedCrossAxisCount
          itemBuilder: (_, i) {
            return GestureDetector(
              onTap: () {
                Navigator.push(
                  context,
                  MaterialPageRoute(builder: (_) => menus[i]["page
```

HOME PAGE

Halaman Home berfungsi sebagai pusat navigasi.
Pada halaman ini ditampilkan beberapa menu gesture dalam bentuk grid (kotak-kotak).
Setiap menu berisi:

- Judul fitur
- Halaman tujuan

Saat salah satu menu ditekan:

- Aplikasi akan berpindah halaman menggunakan Navigator.push()

Komponen penting:

- GridView → menampilkan layout grid
- GestureDetector → menangkap klik pada menu

```
lib > pages > tap_page.dart > _TapPageState
1   import 'package:flutter/material.dart';
2
3   class TapPage extends StatefulWidget {
4     @override
5     _TapPageState createState() => _TapPageState();
6   }
7
8   class _TapPageState extends State<TapPage> {
9     double size = 150;
10    Color color = Colors.blue;
11
12    void randomize() {
13      setState(() {
14        size = size == 150 ? 220 : 150;
15        color = color == Colors.blue ? Colors.red : Colors.b
16      });
17    }
18
```

TAP INTERACTION

Halaman ini menunjukkan interaksi dasar sentuhan.
Jenis gesture yang digunakan:

- Tap → mengubah ukuran dan warna objek
- Double Tap → mengubah warna menjadi hijau
- Long Press → memperbesar ukuran objek

Cara kerjanya:
Saat pengguna menyentuh objek:

- Nilai variabel seperti size dan color berubah
- setState() dipanggil untuk memperbarui tampilan

Widget AnimatedContainer digunakan agar perubahan terlihat halus (animasi).

```
> pages > drag_page.dart > _DragPageState
1   import 'package:flutter/material.dart';
2
3   class DragPage extends StatefulWidget {
4     @override
5     _DragPageState createState() => _DragPageState();
6   }
7
8   class _DragPageState extends State<DragPage> {
9     double x = 100;
10    double y = 100;
11
12    @override
13    Widget build(BuildContext context) {
14      return Scaffold(
15        appBar: AppBar(title: Text("Drag Movement")),
16        body: Stack(
17          children: [
18            Positioned(
19              left: x,
20              top: y,
21              child: GestureDetector(
22                onPanUpdate: (d) {
23                  setState(() {
24                    x += d.delta.dx;
25                    y += d.delta.dy;
26                  });
27                },
```

DRAG MOVEMENT

Halaman ini digunakan untuk menggerakkan objek.

Cara kerjanya:

- Posisi objek disimpan dalam variabel x dan y
- Gesture onPanUpdate digunakan untuk membaca pergerakan jari
- Nilai posisi diperbarui berdasarkan delta (pergeseran)

Widget yang digunakan:

- Stack → untuk layout bebas
- Positioned → untuk menentukan posisi objek

Hasilnya:

Objek dapat digeser ke segala arah sesuai gerakan jari

lib > pages > scale_page.dart > ...

```dart
import 'package:flutter/material.dart';
import 'package:flutter/gestures.dart';

class ScalePage extends StatefulWidget {
  const ScalePage({super.key});

  @override
  State<ScalePage> createState() => _ScalePageState();
}

class _ScalePageState extends State<ScalePage> {
  double scale = 1.0;
  double previousScale = 1.0;

  Offset position = Offset.zero;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Scale & Zoom (PRO)")),
      body: Center(
        child: Listener(
          onPointerSignal: (pointerSignal) {
            // ✅ Zoom pakai scroll (laptop / mouse)
            if (pointerSignal is PointerScrollEvent) {
              setState(() {
                scale += pointerSignal.scrollDelta.dy * -0.001;
                scale = scale.clamp(0.5, 3.0);
```

SCALE & ZOOM

Halaman ini menunjukkan interaksi yang lebih kompleks yaitu zoom dan geser sekaligus.

Gesture yang digunakan:

- onScaleStart → menyimpan skala awal
- onScaleUpdate → mengatur zoom dan posisi

Variabel penting:

- scale → ukuran objek
- position → posisi objek

Fitur tambahan:

- clamp() digunakan untuk membatasi zoom agar tidak terlalu besar/kecil
- onPointerSignal memungkinkan zoom menggunakan scroll mouse (desktop)

Widget:

- Transform.scale → untuk zoom
- Transform.translate → untuk geser

```dart
import 'package:flutter/material.dart';

class PlaygroundPage extends StatefulWidget {
  @override
  _PlaygroundPageState createState() => _PlaygroundPageState();
}

class _PlaygroundPageState extends State<PlaygroundPage> {
  double x = 100, y = 100;

  double scale = 1.0;
  double previousScale = 1.0;

  Color color = Colors.blue;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Gesture Playground")),
      body: Stack(
        children: [
          Positioned(
            left: x,
            top: y,
            child: GestureDetector(
              // TAP
              onTap: () => setState(() => color = Colors.green),
```

GESTURE PLAYGROUND

Halaman ini merupakan gabungan semua gesture.
Gesture yang digunakan:

- Tap → mengubah warna
- Double Tap → mengubah warna berbeda
- Long Press → memperbesar objek
- Drag → menggeser objek
- Scale → zoom objek

Semua gesture digabung dalam satu GestureDetector.

Cara kerja:

- Setiap interaksi mengubah nilai variabel (warna, posisi, skala)
- setState() digunakan untuk memperbarui tampilan

Halaman ini menunjukkan bagaimana gesture bisa digabung dalam satu objek

KESIMPULAN

Aplikasi Gesture Lab ini menunjukkan bagaimana berbagai jenis gesture di Flutter, seperti tap, drag, dan scale (zoom), dapat digunakan untuk menciptakan interaksi yang dinamis pada aplikasi. Setiap gesture dideteksi menggunakan GestureDetector, kemudian perubahan yang terjadi pada objek seperti warna, ukuran, dan posisi diatur melalui variabel state dan diperbarui secara langsung menggunakan setState(). Semakin kompleks gesture yang digunakan, seperti kombinasi drag dan zoom dalam satu objek, maka semakin kompleks pula logika yang diperlukan, terutama dalam pengolahan posisi dan skala secara bersamaan, sehingga aplikasi menjadi lebih interaktif dan responsif terhadap input pengguna.

Terima Kasih